

Bayesian Modeling of Human Cervical Spine Motion Segments Under Pure Shear Loading

Stat 447C Final Project

Luis Dias 21074117

April 2025

Introduction

Fracture-Dislocations (FDs) of the cervical spine are serious injuries that cause significant trauma to the spinal cord, often resulting in severe paralysis or death. FDs occur primarily from head-first impact scenarios, such as vehicle rollovers and sports accidents, and *ex vivo* inertial impact studies have indicated that there is a significant shearing component to the internal loads in the cervical spine during such an injury (1–4). Despite this, most segment-level testing in the literature has been on bending moments and compression rather than pure shear loading. This lack of data in shear represents a gap in our understanding of an important mechanism of injury in the spine, which, in turn, hampers efforts to validate computer models of injury in the human spine, and complicates the development of strategies to mitigate and protect against FD injuries (5–7).

As part of my MASc, I am performing shear tests on pairs of human vertebrae to determine their mechanical characteristics under pure shear loading. A basic schematic of the test setup is shown in Fig. 1. Each specimen is obtained from an individual donor, and 6 non-destructive displacement-controlled tests are performed: in 2 shearing directions (along the anterior and posterior axes) and 3 displacement rates (1, 10, and 100 mm/s). The shear load in the specimen is recorded for analysis.

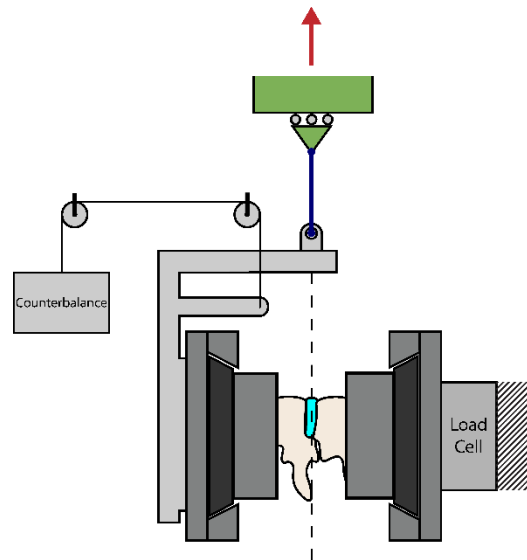


Figure 1 - Basic schematic of non-destructive shear test setup

In the tests carried out so far, I have observed a non-linear (roughly second-order) response of the shearing force to constant-rate shearing displacements. Rather than simplifying my data through linearization, I would like to capture this non-linearity in a Bayesian model.

More specifically, my goal is to set up the basic scaffolding for a quadratic regression model of my data, capturing the effects of mechanical and intrinsic covariates (see Table 1), and including subject-specific random effects. This model should be capable of inferring the expected response and 95% credible interval of future tests, given a displacement vector and test-specific covariates.

Bayesian approaches have been successfully employed in biomechanical modeling successfully in recent literature (8–10), and I believe they would work well for estimating over the parameter space in my research.

Model construction

Covariates

Table 1 shows the test covariates used in the development of this model. Some covariates (demographic and anthropometric) are intrinsic to each specimen and do not change between each test of that specimen. Each of the six tests for a given specimen involves a unique combination of the mechanical factors (rate and direction).

Table 1 - Covariates used in model construction

Category	Covariate	Variable	Support
Mechanical (test-level)	Direction	d	Binary, $d \in \{-1,1\}$
	Rate	r	Continuous, $r \in \mathbb{R}^+$
Demographic (specimen-level)	Age	yr	Continuous, $yr \in \mathbb{R}^+$
	Sex	sx	Binary, $sx \in \{-1,1\}$
Anthropometric (specimen-level)	Bone Volume Fraction (BV/TV)	bv	Continuous, $bv \in [0,1]$
	Disc Degeneration Grade	dd	Discrete, over $[0,7]$. Modeled as continuous, $dd \in \mathbb{R}^+$
	Facet Joint Degeneration Grade	fjd	Discrete, over $[0,4]$. Modeled as continuous, $fjd \in \mathbb{R}^+$

Mathematical Formulation

The basic setup for sampling each datapoint y is:

$$y_{s,d,r,i} \sim \text{Norm}(\mu_{s,d,r,i}, \sigma)$$

Where:

- s : specimen
- d : direction
- r : rate
- i : datapoint index for a given test

This captures the inherent measurement noise in the system, which is expected to be constant across the measurement range of the load cell.

The mean parameter, μ , is modeled as a second-order polynomial function:

$$\mu_{s,d,r,i} = A \cdot x_{s,d,ri}^2 + B \cdot x_{s,d,ri}$$

Where A and B are linear mixed effects terms, with one overall fixed factor (a_{all}), fixed factors for each covariate (given by $a_{(covar)}$ and $b_{(covar)}$) and random intercepts ψ_s and ϕ_s for each specimen, to capture inherent variability between individuals. No quadratic intercept was implemented, as the data was intentionally normalized such that both force and displacement start at 0.

$$A = a_{all} + a_{rate} \cdot r + a_{dir} \cdot d + a_{age} \cdot yr + a_{sex} \cdot sx + a_{bv/tv} \cdot bv + a_{Ddeg} \cdot dd + a_{FJdeg} \cdot fjd + \psi_s$$

$$B = b_{all} + b_{rate} \cdot r + b_{dir} \cdot d + b_{age} \cdot yr + b_{sex} \cdot sx + b_{bv/tv} \cdot bv + b_{Ddeg} \cdot dd + b_{FJdeg} \cdot fjd + \phi_s$$

Random intercepts were obtained from zero-centered normal priors with hierarchical standard deviations, themselves sampled from a non-centered normal prior:

$$\sigma_a, \sigma_b \sim \text{Norm}(1,1)$$

$$\psi_s \sim \text{Norm}(0, \sigma_a)$$

$$\phi_s \sim \text{Norm}(0, \sigma_b)$$

Finally, every covariate factor was given a normal prior, the parameters of which were fine-tuned during the prior-predictive check, ultimately arriving at values of 30 and 15 for the mean and SD:

$$a_{all}, a_{(covar)}, b_{all}, b_{(covar)} \sim \text{Norm}(30, 15)$$

Implementation

The model was implemented in Stan (Version 2.32.2) and data cleanup and analysis was performed in Rstudio (Version 2024.12.0) with R Version 4.4.2. For speed, data was downsampled by a factor of 10 prior before being passed to Stan. All data was normalized and centered prior to analysis.

Analysis

Prior Predictive Check

Priors were visually verified by simulating in Stan without true observations, and plotting the results against the true test data. Intercepts were on hierarchical priors, but the prior parameters for covariate factors were tuned manually to ensure prior behaviour was reasonable. Priors were chosen such that random draws covered a similar area of the first quadrant as the observed data. Figure 2 shows an example of the desired prior behaviour during tuning.

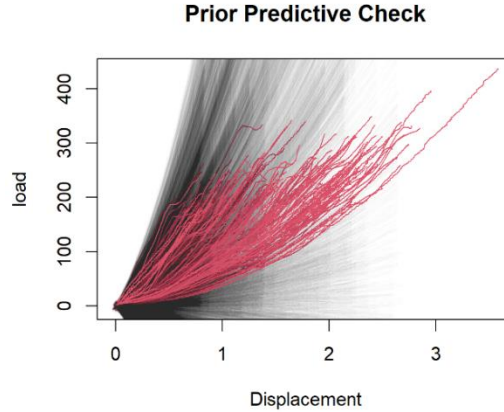


Figure 2 - Prior predictive check over 200 iterations (black), with the observed data overlaid (red)

Posterior Predictive Check (Pseudo-LOO)

A posterior predictive check was implemented in Stan using generated quantities. This was a pseudo-Leave-One-Out (LOO) check – the model was trained on the full dataset, and used to predict the force vector (along with 95% CIs) for each of the tests. Each true vector was then compared to the CI, and the proportion of predicted points that falls within the interval was calculated.

The choice of a pseudo- vs. true LOO (where the training dataset excludes the left-out data) was motivated mainly by computational limitations – due to the large dataset, fitting the model can take a very long time, and re-fitting over several hundred tests becomes impractical. A true LOO may be possible after more optimization of the model (see Future Work).

The pseudo-LOO coverage with 1000 iterations was 0.9025, below the nominal coverage of 95%. The true coverage is likely lower than this, as the pseudo-LOO training set included the observed test.

A test with synthetic data (200 iterations; coverage = 0.9517) confirmed that this under-coverage was not due to a bug, but was likely due to either model mis-specification or slow mixing. Mixing appeared normal according to a trace plot analysis, but this was done on a reduced simulation of 400 iterations and 2 chains (as larger simulations were computationally impractical), so slow mixing cannot be fully eliminated as a culprit until a faster model is implemented (see Future Work).

Proof-of-concept Inference

A custom inference engine was implemented in R, taking as input the samples generated by my Stan model, and a set of test parameters (displacement vector and covariates) for a queried test. It returns an array of predicted force vectors (one per iteration in the stan output), which can be used to generate an expected response and 95% CI. Figure 3 shows an example plot for a hypothetical 70-y.o. male subject, with a BV/TV of 0.6, disc and facet joint degeneration grades of 4 and 2, in anterior shear at 10 mm/s. Note that this is a proof of concept, and cannot be used in practice until true coverage is identified and the model is properly calibrated.

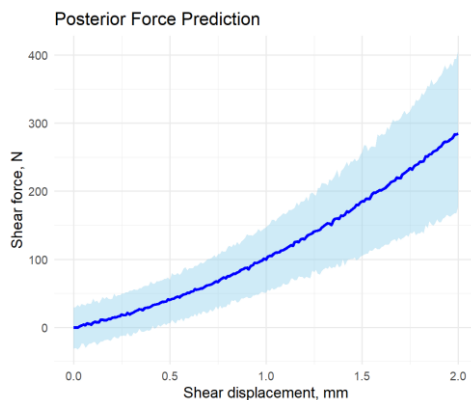


Figure 3 - Sample inference for an unobserved hypothetical test

Limitations

The primary limitation are the long runtimes, which restrict my maximum iterations and potentially contribute to the observed under-coverage. The use of a pseudo-LOO also means that my true coverage is lower than my current estimate. Finally, while I have approximated the behaviour as second-order polynomial, it may be better modeled by another function (for example, an exponential).

Conclusion and Future Work

This model is still in the proof-of-concept stage, but shows promise for my research. More work needs to be done to improve runtimes and increase the feasible iteration count. I intend to try the same model in Cmdstanr, Rstanarm, and Brms, to potentially improve runtime and allow a more thorough check of the mixing behaviour and computation of bigger models. I will also explore a reformulation of my data to improve computational efficiency in Stan, by reducing the dimensionality of my input matrix, or further downsampling the data (a suboptimal tradeoff for computational efficiency).

Additionally, I intend to implement a true Leave-One-Subject-Out (LOSO) check, where all the tests for one subject are left out of the training set and predicted. I already have a functioning implementation of this based in my R script, but it is not yet practical due to the long runtimes.

See <https://github.com/luidias/STAT447C-Project> for the full file set, including the R script containing my analysis and custom inference code.

Bibliography

1. Nightingale RW, Bass CR, Myers BS. On the relative importance of bending and compression in cervical spine bilateral facet dislocation. *Clinical Biomechanics*. 2019 Apr 1;64:90–7.
2. Ivancic PC, Pearson AM, Tominaga Y, Simpson AK, Yue JJ, Panjabi MM. Biomechanics of cervical facet dislocation. *Traffic Injury Prevention*. 2008 Dec;9(6):606–11.
3. Bauze RJ, Ardran GM. Experimental production of forward dislocation in the human cervical spine. *J Bone Joint Surg Br*. 1978;60-B(2):239–45.
4. Roaf R. A study of the mechanics of spinal injuries. *The Bone and Joint Journal*. 1960;42-B(4):810–23.
5. Whyte T, Barker JB, Cronin DS, Dumas GA, Nolte LP, Cripton PA. Load-Sharing and Kinematics of the Human Cervical Spine Under Multi-Axial Transverse Shear Loading: Combined Experimental and Computational Investigation. *Journal of Biomechanical Engineering* [Internet]. 2021 Mar 17 [cited 2021 Apr 13];143(061013). Available from: <https://doi.org/10.1115/1.4050030>
6. Barker JB, Cronin DS, Nightingale RW. Lower Cervical Spine Motion Segment Computational Model Validation: Kinematic and Kinetic Response for Quasi-Static and Dynamic Loading. *Journal of Biomechanical Engineering* [Internet]. 2017 May 2 [cited 2024 Dec 1];139(061009). Available from: <https://doi.org/10.1115/1.4036464>
7. Singh D, Cronin DS. An investigation of dimensional scaling using cervical spine motion segment finite element models. *International Journal for Numerical Methods in Biomedical Engineering*. 2017;33(11):e2872.
8. Santos VJ, Valero-Cuevas FJ. A Bayesian approach to biomechanical modeling to optimize over large parameter spaces while considering anatomical variability. In: *The 26th Annual International Conference of the IEEE Engineering in Medicine and Biology Society* [Internet]. San Francisco, CA, USA: IEEE; 2004 [cited 2025 Apr 15]. p. 4626–9. Available from: <http://ieeexplore.ieee.org/document/1404282/>
9. Ma HT, Yang Z, Griffith JF, Leung PC, Lee RY, W. A new method for determining lumbar spine motion using Bayesian belief network. *Medical and Biological Engineering and Computing*. 2008 Apr;46(4):333–40.
10. Johnson RT, Lakeland D, Finley JM. Using Bayesian inference to estimate plausible muscle forces in musculoskeletal models. *Journal of NeuroEngineering and Rehabilitation*. 2022 Mar 23;19(1):34.

Appendix – Stan Code

```
// Biomechanical Spine Segments in Shear
// Stat 447C Final Project
// Luis Dias
// April 2025

data {
  // raw data
  int<lower=0> N; // number of observations in each vector
  int<lower=0> K; // number of tests
  matrix[K, N] y; // Shear force data
  matrix[K, N] x; // Shear displacement data
  int<lower = 1, upper = N> L[K]; // X and Y data is ragged so I padded it for stan - this vector contains the actual data lengths for each test.

  // test parameters (predictors)
  matrix[K, 8] t_pars; // test parameter matrix. Columns are [ID, dir, log(rate), age, sex, BMD, DD, FJD]
  int<lower=0> S; // number of subjects
  int<lower=1,upper=S> subject ids[K]; // corresponding subject ID for each test
}

parameters {
  // LMM terms for first and second-order parameters of the quadratic regression
  // 2nd-order params
  real a_all; // "overall" second order term without interactions from covars
  real a_dir;
  real a_rate;
  real a_age;
  real a_sex;
  real a_bv;
  real a_dd;
  real a_fjd;
  // 1st-order params
  real b_all;
  real b_dir;
  real b_rate;
  real b_age;
  real b_sex;
  real b_bv;
  real b_dd;
  real b_fjd;

  real<lower=0> sigma; // measurement noise. global for now, change if needed.

  // params for hierarchical model of random intercepts
  vector[S] z_a; // normalized z scores (see model block)
  vector[S] z_b;
  real<lower=0> sig_a; // scaling parameters for z scores
  real<lower=0> sig_b;
}

transformed parameters{
  vector[S] a_intercepts = z_a * sig_a; // vector of random intercepts for the second-order term, one per subject
  vector[S] b_intercepts = z_b * sig_b; // same as above, for first-order term
}

model {
  // tuning prior parameters - note that these will be repeated 9 times per term
  real a_priorMu = 30;
  real a_priorVar = 15;
  real b_priorMu = 30;
  real b_priorVar = 15;

  // priors - weakly informative
  a_all ~ normal(a_priorMu, a_priorVar);
  a_dir ~ normal(a_priorMu, a_priorVar);
  a_rate ~ normal(a_priorMu, a_priorVar);
  a_age ~ normal(a_priorMu, a_priorVar);
  a_sex ~ normal(a_priorMu, a_priorVar);
  a_bv ~ normal(a_priorMu, a_priorVar);
  a_dd ~ normal(a_priorMu, a_priorVar);
  a_fjd ~ normal(a_priorMu, a_priorVar);
  b_all ~ normal(b_priorMu, b_priorVar);
  b_dir ~ normal(b_priorMu, b_priorVar);
  b_rate ~ normal(b_priorMu, b_priorVar);
  b_age ~ normal(b_priorMu, b_priorVar);
  b_sex ~ normal(b_priorMu, b_priorVar);
  b_bv ~ normal(b_priorMu, b_priorVar);
  b_dd ~ normal(b_priorMu, b_priorVar);
  b_fjd ~ normal(b_priorMu, b_priorVar);
  sigma ~ exponential(1);

  // priors for hierarchical subject intercepts
  z_a ~ normal(0, 1); // centered at zero and std dev of 1
  z_b ~ normal(0, 1);
  sig_a ~ normal(1, 1); // hyperprior scaling factor for intercept
  sig_b ~ normal(1, 1);

  //vectorize to speed things up
  matrix[K, 7] X_test = t_pars[, 2:8]; // predictors only
  vector[K] a_fixed = a_all + X_test * vector({a_dir, a_rate, a_age, a_sex, a_bv, a_dd, a_fjd});
  vector[K] b_fixed = b_all + X_test * to_vector({b_dir, b_rate, b_age, b_sex, b_bv, b_dd, b_fjd});

  //likelihood (vectorized in slices, one per test)
  for (k in 1:K) {
    // slice out the tests's x and y data, excluding any padding
    vector[L[k]] x_k = to_vector(x[k, 1:L[k]]); // displacement for test k, using only indices up to the padding (1:L[k])
    vector[L[k]] y_k = to_vector(y[k, 1:L[k]]); // force for same test

    vector[L[k]] mu_k = (a_fixed[k] + a_intercepts[subject ids[k]]) * x_k .* x_k
      + (b_fixed[k] + b_intercepts[subject ids[k]]) * x_k;
  }
}
```

```

    // likelihood calculation. Comment this out when doing prior predictive, leave it in for (basic) posterior predictive
    y_k ~ normal(mu_k, sigma);
  }
}

generated quantities{
  // comment this whole thing out unless you're doing predictive checks with stan. It's slow.

  // predictive check. Comment out the y_k ~ line in the model block to perform a prior check, leave it uncommented for a posterior.
  // note that this posterior check is a pseudo-leave-one-out. We train on the full dataset and try to re-predict a datapoint.
  // A true leave-one-out should be implemented later, outside of stan

  matrix[K, N] y_pp; // simulated force data

  // everything in this loop is functionally identical to the model above, but real data is ignored
  for (k in 1:K) {
    vector[L[k]] x_k = to_vector(x[k, 1:L[k]]);

    // only use one of these pairs
    real a_int = normal_rng(0, sig_a); // true new intercept
    real b_int = normal_rng(0, sig_b);
    //real a_int = a_intercepts[subject_ids[k]]; // reuse subject intercepts for leave-one-test out
    //real b_int = b_intercepts[subject_ids[k]];

    vector[L[k]] mu_k = (a_all
      + a_dir*t pars[k,2] + a_rate*t pars[k,3]
      + a_age*t pars[k,4] + a_sex*t pars[k,5]
      + a_bv*t pars[k,6] + a_dd*t pars[k,7] + a_fjd*t pars[k,8]
      + a_int)
      * x_k .* x_k
    + (b_all
      + b_dir*t pars[k,2] + b_rate*t pars[k,3]
      + b_age*t pars[k,4] + b_sex*t pars[k,5]
      + b_bv*t pars[k,6] + b_dd*t pars[k,7] + b_fjd*t pars[k,8]
      + b_int)
      * x_k;

    // simulate y up the the test length
    for (n in 1:L[k]){
      y_pp[k,n] = normal_rng(mu_k[n],sigma);
    }

    // pad remaining length
    for (n in L[k]+1:N){
      y_pp[k,n] = 0; // would prefer Infs or NAs but I haven't figured out how to declare them in stan
    }
  }
}

```